

历史 OD 驱动的选择性区域物化： 面向精确最短路查询的无参数需求传播与神经排序残差

席圣洋（第一作者）

肖逸涵（第二作者）

王加文（第二作者）

初稿日期：2026 年 8 月 1 日

作者单位：_____

通讯作者：_____ 电子邮箱：_____

的资源配置。

关键词：道路网络；最短路索引；历史 OD；选择性区域物化；图扩散；学习排序

摘要

道路网络分析和交通仿真需要反复执行大规模起终点（origin-destination, OD）最短路查询。全图在线搜索开销较高，而全量索引预处理又会占用大量存储。本文在已有 CRP 式边界覆盖图精确索引之上，研究一个更具体的问题：如何仅根据历史 OD 与静态道路图，在固定预算下选择少量值得物化的连通区域，同时保持在线查询距离完全精确。为此，本文构造全图空间均匀候选和基于真实查询展开节点减少量的精确区域收益标签，并采用历史、标签和未来三个互不重叠的连续时间窗口，以及基于 Jaccard 重叠连通组的空间隔离。本文首先提出面向该任务的无参数方法 Z0：分别将历史起点和终点边际需求沿有向道路图正、反方向进行固定 lazy 扩散，在多尺度上聚合区域需求暴露。在此基础上，G4 采用 32 层双塔图传播网络学习冻结 Z0 无法解释的排序残差，并以可微 Spearman 目标和仅依赖验证集的峰值保存协议稳定训练。Porto 与 Chicago 两城实验表明，G4 在“三种子 × 两城 × 两外部窗口”的 12 次配对比较中全部提高全局 Spearman，显示出跨空间与时间切分的一致神经增量。与此同时，Top- K 指标呈现独立变化，实证证明全局顺序一致性与固定预算下的头部选择是两个不可互相替代的学习目标。严格非重叠部署下，两城全部在线查询均保持 100% 距离正确。C++ 同实现评测进一步表明，Porto 的平均查询耗时下降 6.51%–7.48%，而 Chicago 的收益主要体现为展开节点和多数 P95 尾延迟下降，平均耗时受密集 shortcut 扫边开销限制。结果说明，历史需求传播能够为精确最短路索引提供稳健、可解释的区域资源配置依据；神经残差则提供了可重复验证的全局排序增益。完整候选序列能够服务预算变化、区域互斥和候补选区，使系统从只追逐固定 Top- K 热点转向顾及全图需求-收益结构

1 引言

最短路距离查询是交通仿真、出行需求分析、位置服务和道路网络研究中的基础操作。经典 Dijkstra 算法能够给出精确答案，但当同一城市路网上持续到来大量 OD 查询时，重复搜索会造成显著计算开销。已有研究通过层次化预处理、标签索引或区域覆盖图减少在线工作量 [2, 4]。这些方法通常面向整张图构建索引，而真实查询负载往往具有明显的空间偏斜和时间局部性：有限预算下，并非所有道路区域都值得获得相同的预处理资源。

负载感知最短路研究已经证明，历史查询分布可用于调整精确索引 [1, 14]；查询缓存工作也利用历史日志估计可复用对象的价值 [8, 12]。然而，标签索引、路径缓存和区域边界覆盖图对应不同的物化对象与成本结构。本文沿用底层边界 shortcut，并把问题聚焦到：在 CRP 式精确覆盖图底座上，只物化少量候选区域时，应如何根据历史 OD 对候选进行排序，并在时空隔离、区域互斥和真实在线查询约束下验证该排序。

这一问题具有三个困难。第一，若用历史最短路径直接统计区域命中，会把昂贵的路径计算提前放入输入阶段，也容易使方法退化为路径缓存。第二，候选区域之间可能高度重叠；随机划分会让近重复区域跨越训练集和测试集，造成严重空间泄漏。第三，离线相关性、单区域收益和多区域在线加速并不等价：重叠候选无法同时物化，shortcut 数量和端点接入还会改变实际查询成本。

本文将 AI 限定为离线资源配置工具。模型不预测最短路，也不改变距离答案；在线阶段仍由精确双向 Dijkstra 和区域局部接入完成查询。本文的主要贡献如下：

1. 在既有负载感知精确索引和 CRP 式边界覆盖图基

础上,系统定义历史 OD 驱动的选择性区域物化任务,以真实展开节点减少量作为候选区域价值,并保持学习选区与精确查询解耦。

2. 构建无历史路径输入的无参数双向多尺度需求传播组合 Z0,并通过两城的方向、拓扑、需求来源、深度和池化正交消融解释其有效信号来源。
3. 构建以 Z0 为冻结先验的 32 层神经排序残差 G4。该方法采用稳定的严格恒等残差路径和与评价一致的 soft Spearman 目标,在 Porto 与 Chicago 的多种子空间留出集和未来窗口上实现 12/12 次配对全局排序增益,证明神经模型稳定学习到确定性先验之外的有效信息。
4. 建立 H/Y/F 时间隔离、Jaccard 空间重叠组隔离、严格非重叠部署和精确在线配对的验证闭环,通过全局 Spearman 与 Top-K 指标的可重复分离,实证证明全局排序和预算约束头部选择属于不同学习目标。完整排序为动态预算、重叠淘汰和全图候补提供可复用依据;C++ 同实现评测进一步揭示展开节点、shortcut 密度和真实延迟之间的系统关系。

围绕上述贡献,实验集中回答五个研究问题: RQ1,历史需求传播能否在空间隔离候选上预测区域价值; RQ2,冻结排序能否推广到未参与建模的未来窗口; RQ3, Z0 的收益究竟来自历史需求、正确道路拓扑、传播方向还是简单池化统计; RQ4,单区域排序能否在严格不重叠的多区域部署中转化为精确查询工作量收益; RQ5,同实现下的工作量减少能否进一步转化为真实墙钟加速。本文按此顺序组织实验,避免用单一相关系数替代系统结论。

2 相关工作

2.1 精确最短路索引与区域覆盖图

道路最短路加速包含层次化搜索、标签索引和基于分区的覆盖图等路线。SALT 将图分区、边界覆盖图与查询算法组合到统一框架中 [4]。其中,对区域内部运行受限最短路、在边界节点之间建立精确 shortcut、删除内部节点后在覆盖图上查询,与本文的底层压缩结构高度一致。因此本文明确把该部分视为已有索引底座,不将其列为原创算法。本文的研究对象是只选择少量区域物化时的候选排序、内部端点接入、严格不重叠部署和同预算在线效果。

2.2 负载感知索引与最短路缓存

WCF 利用道路查询负载的空间偏斜与时间局部性构造 workload-aware Core-Forest 标签索引 [14]; FAHL

进一步研究 flow-aware 的标签索引 [1]。另一些工作从历史查询日志中选择值得缓存的路径或区域对象 [8, 12]。这些工作说明“历史负载驱动精确最短路加速”并非新问题。与之不同,本文不预测高频节点后构建 2-hop/Core-Forest 标签,也不缓存已出现的完整路径,而是对固定、空间均匀的连通区域预测其精确展开收益,并在 CRP 式覆盖图中进行选择性地物化。

2.3 图扩散、深层传播与可微排序

DCRNN 在有向道路图上使用双向随机游走建模空间依赖 [9]; SIGN 和 Jumping Knowledge 展示了多尺度图扩散和跨层表示读出的价值 [5, 15]。因此,本文不把 Z0 的正反向多步传播写成新的基础扩散算子,而强调其在“起点/终点边际注入-固定 lazy 扩散-区域 mean+max-精确收益排序”上的任务化组合。

G4 的传播组织受到 NBFNet 的 Indicator-Message-Aggregate 框架启发 [16]。深层网络中的 pre-activation 残差和小尺度残差已有明确先例 [7, 13],可微排序亦是成熟方向 [6]。本文不将这些组件分别作为新通用架构,而研究它们能否在强确定性先验上形成稳定、可审计的任务增量。

3 问题定义

3.1 道路图、查询负载与时间窗口

给定带正权有向道路图 $G = (V, E, w)$, 其中 $w: E \rightarrow \mathbb{R}_{>0}$ 表示道路长度。一个查询 $q = (o_q, d_q, t_q)$ 包含起点、终点和时间戳。全部查询按时间排序后划分为三个连续且互不重叠的窗口: 历史窗口 $H = [0, 0.35)$ 、当前标签窗口 $Y = [0.35, 0.70)$ 和未来冻结窗口 $F = [0.70, 1.00)$ 。H 只用于构造需求输入, Y 用于生成训练标签和 train/validation 选型, F 只在协议冻结后进行零样本时间评测。

历史窗口仅保留起终点边际需求。对节点 v , 定义

$$\begin{aligned} h_O(v) &= \frac{1}{|H|} \sum_{q \in H} \mathbf{1}[o_q = v], \\ h_D(v) &= \frac{1}{|H|} \sum_{q \in H} \mathbf{1}[d_q = v]. \end{aligned} \quad (1)$$

方法不读取 H 中任一查询的最短路径。

3.2 候选区域与精确收益标签

候选集合 $\mathcal{R} = \{R_1, \dots, R_M\}$ 在全图节点上以固定随机种子尽量空间均匀地产生,每个候选是包含 512 个节点的连通子图。候选生成不读取历史 OD、标签或模

图待补

方法总图：H/Y/F 时间轴、候选区域、Z0、G4、hard-disjoint 选择、CRP 式覆盖图和精确查询。

图 1: 本文端到端流程。底层覆盖图采用已有精确索引结构, 本文研究重点是历史 OD 驱动的区域排序、隔离与选择性物化。

型输出。区域 R 的边界集合 B_R 包含与区域外节点相邻的节点, 其余节点构成内部集合 I_R 。

对标签查询 q , 记原图双向 Dijkstra 的展开节点数为 $N_G(q)$, 只物化区域 R 后的覆盖图搜索与端点局部接入总展开节点数为 $N_R(q)$ 。区域的监督标签定义为

$$y_R^Y = \frac{1}{|Q_Y|} \sum_{q \in Q_Y} [N_G(q) - N_R(q)], \quad (2)$$

其中 Q_Y 是从 Y 中以预先固定随机种子抽取的 2,000 条查询。未来标签 y_R^F 使用完全相同定义, 但仅在最终评测时从 F 生成。标签衡量精确算法工作量而非不稳定的单次 Python 墙钟时间; 每条查询同时校验索引距离与原图距离相等。

3.3 选择性物化目标

排序器输出候选分数 s_R 。部署阶段按分数从高到低贪心选择 $K \in \{5, 10, 18\}$ 个区域, 且要求任意两个已选区域节点集合严格不相交:

$$|S| = K, \quad R_i \cap R_j = \emptyset, \quad \forall R_i \neq R_j \in S. \quad (3)$$

这一 hard-disjoint 约束来自当前索引的物化语义。式(2)是单区域边际收益, 多区域收益不能简单相加, 因此最终价值必须通过将整个 S 一次性物化后的真实在线查询确定。

4 方法

4.1 CRP 式选择性区域物化

对每个被选择区域 R , 离线阶段从每个 $u \in B_R$ 出发, 在诱导子图 $G[R]$ 上运行受限 Dijkstra, 并对每个可达的 $v \in B_R$ 建立有向 shortcut:

$$w_R(u, v) = d_{G[R]}(u, v). \quad (4)$$

随后删除 I_R , 保留外部节点、边界节点、剩余原始边和 shortcut。该结构沿用已有 CRP/边界覆盖图思想 [4]。

若查询端点位于某个 I_R , 算法只在该区域内部计算端点至边界的精确距离, 并将这些距离作为双向搜索的初始 frontier; 若起终点位于同一区域, 还比较区域内部直达距离。由此避免整条查询回退原图, 同时保持精确性。

距离精确性。 任意穿越区域 R 的原图路径都可按进入和离开 R 的边界节点切分。将其中每段区域内子路径替换为对应 shortcut, 不会增加路径长度, 因为 shortcut 保存的是 $G[R]$ 内的最短距离; 反过来, 每条 shortcut 都对应 $G[R]$ 中一条真实路径, 因而覆盖图不可能产生比原图更短的非合法距离。内部端点通过区域内精确 frontier 接入, 起终点同区时另行比较区内直达路径。因此在已选区域互不重叠、shortcut 精确且边权非负的条件下, 覆盖图查询与原图最短路径距离相等。实现仍逐查询进行数值校验, 不以理论论证替代工程正确性测试。

预处理与查询成本。 设区域 R 的边界数为 b_R 、内部诱导边数为 m_R 。直接从每个边界节点运行一次受限 Dijkstra, 预处理复杂度约为 $O(b_R(m_R + |R|) \log |R|)$, 潜在 shortcut 数为 $O(b_R^2)$ 。因此相同的 512 节点预算并不意味着相同的索引成本: 边界更稠密的区域会产生更多 shortcut 和在线扫描。本文同时报告区域节点预算、shortcut 数、展开节点与扫描边, 正是为了避免把节点数相同误认为系统成本相同。

4.2 无参数双向多尺度需求场 Z0

首先对历史边际需求进行对数缩放:

$$\mathbf{o}^{(0)} = \log(1 + 1000\mathbf{h}_O), \quad \mathbf{d}^{(0)} = \log(1 + 1000\mathbf{h}_D). \quad (5)$$

令 $P_{\rightarrow}$ 表示沿原图方向、按接收节点归一化的均值传播矩阵, $P_{\leftarrow}$ 表示在反图上的对应传播矩阵。每一层采用带自环的固定 lazy mean:

$$\begin{aligned} \mathbf{o}^{(k+1)} &= \frac{1}{2}(\mathbf{I} + P_{\rightarrow})\mathbf{o}^{(k)}, \\ \mathbf{d}^{(k+1)} &= \frac{1}{2}(\mathbf{I} + P_{\leftarrow})\mathbf{d}^{(k)}. \end{aligned} \quad (6)$$

在深度集合 $\mathcal{D} = \{1, 2, 4, 8, 16, 32\}$ 上, 区域分数为

$$z_R = \frac{1}{|\mathcal{D}|} \sum_{k \in \mathcal{D}} \left[\text{mean}_{v \in R} e_v^{(k)} + \max_{v \in R} e_v^{(k)} \right], \quad (7)$$

$$\mathbf{e}^{(k)} = \mathbf{o}^{(k)} + \mathbf{d}^{(k)}.$$

Z0 不含可学习参数, 不读取 Y 或 F 。其目标不是预测路径, 而是估计候选区域在多尺度道路邻域中暴露于历史起终点需求的程度。

4.3 以 Z0 为先验的神经排序残差 G4

G4 包含起点塔和终点塔, 二者分别沿正图和反图传播且参数不共享。每层使用边特征门控的 message-aggregate 分支 $\mathcal{F}_\ell$, 并采用严格恒等的 pre-norm 小残差更新:

$$\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + 0.01 \mathcal{F}_\ell(\text{LayerNorm}(\mathbf{h}^{(\ell)}), E). \quad (8)$$

该结构用于抑制旧 32 层主干中的反向指数放大; 它是已有深层残差思想在本任务中的稳定化实现, 不是新的通用残差结构。模型在 1, 2, 4, 8, 16, 32 层读取双塔状态并进行区域 mean+max 池化, 输出残差 $r_\theta(R)$ 。最终分数为

$$s_R = z_R + \alpha r_\theta(R), \quad \alpha \in \{0, 0.125, 0.25, 0.5, 1\}. \quad (9)$$

$\alpha = 0$ 严格回退到 Z0; 每个 epoch 的 α 仅在 validation 上选择, 同分时优先更小残差。

4.4 与评价一致的 soft Spearman 目标

旧 pairwise softplus 目标可通过放大分数尺度持续下降, 却不保证真实 Spearman 同向变化。为减少这种目标错位, 先将批内分数标准化为 $\hat{s}_i$, 再用成对 sigmoid 构造软排名 [6]:

$$\tilde{r}_i = \sum_j \sigma\left(\frac{\hat{s}_i - \hat{s}_j}{\tau}\right), \quad \tau = 0.1. \quad (10)$$

设真实标签的平均秩为 r_i^* , 训练损失为

$$\mathcal{L}_{\text{rank}} = 1 - \text{Pearson}(\tilde{\mathbf{r}}, \mathbf{r}^*). \quad (11)$$

所有模型以 validation Spearman 选择 checkpoint。hold-out 和 F 不参与结构、学习率、epoch、 α 或策略选择。

5 实验设计

5.1 数据集与图规模

Porto 使用公开出租车轨迹数据 [10], 将有效轨迹首末点映射为 OD; Chicago 使用 Divvy 2022 年 6 月

共享单车行程 [3]。Chicago 数据先按字段合法性、时间顺序、研究边界和路网吸附质量清洗, 再按完整时间序列等间隔抽取 100,000 条, 不依据模型或区域收益筛选。道路网络来自 OpenStreetMap[11]; Chicago 固定使用与行程时间接近的 2022 年 1 月 Illinois 历史快照, 而不是按实验结果改用最新路网。每城生成 1,200 个 512 节点候选, 并划分为 840/180/180 个 train/validation/holdout 候选。对候选两两计算 Jaccard 重叠, 当重叠不低于 0.50 时连边, 再将整个连通组分配到同一 split。Porto 与 Chicago 分别形成 521 和 452 个重叠组, 跨 split 不存在 Jaccard 不低于 0.50 的候选对。

每个城市的 Y 与 F 都对全部 1,200 个候选、各 2,000 条固定查询生成标签, 因此两城共包含 960 万次候选-查询标签评估。候选、查询抽样种子和 split 在读取 holdout/future 结果前冻结。

5.2 泄漏控制与冻结顺序

候选区域是空间对象, 相邻 BFS 区域常共享大量节点。早期随机 candidate split 中, 直接用训练集最近重叠区域的标签预测测试候选, Spearman 可达约 0.9936, 表明普通随机划分会严重高估泛化。本文因此先建立阈值 0.50 的 Jaccard 重叠图, 再以连通组为最小分配单元。该规则比只删除完全重复区域更严格, 也避免模型通过共享大部分节点“记住”测试区域。

时间上, H 、 Y 和 F 的查询 ID 与时间范围均无交叉。候选身份在读取 H 前由静态道路图确定; 模型输入只来自 H ; 训练与 validation 标签来自 Y ; holdout 只用于空间外部评测; F 在结构、超参数、种子集合和模型选择策略全部冻结后才解锁。Chicago 只复用 Porto 已确定的协议, 不依据其 holdout、future 或在线结果重新调整深度、学习率、温度与残差门。

5.3 对照方法

本文比较以下方法: 随机排序; 按历史端点频率选择的热点方法; 只使用区域几何中心位置的 midpoint Proxy; 无区域传播的 MLP; 无参数 Z0; 以及神经残差 G4。MLP 报告 5 个随机种子, G4 报告 42/43/44 三个种子。所有方法共享相同候选池和部署预算, 不允许根据 holdout 或 future 删除不利种子。

其中随机与历史热点回答“是否仅靠任意区域或端点频率即可获益”; midpoint Proxy 检验简单空间位置是否已经足够; MLP 检验在不传播道路邻接时监督区域特征能够达到的水平; Z0 检验固定图扩散; G4 则只检验监督残差能否超过强先验。Oracle 只用于描述标签空间上界, 不作为可部署方法, 也不参与任何超参数

表 1: 两城数据与冻结切分。

城市	交通数据	节点数	有向边数	全部 OD	H	Y	F
Porto	出租车	133,839	221,589	98,082	34,328	34,329	29,425
Chicago	共享单车	108,114	200,155	100,000	35,000	35,000	30,000

选择。

5.4 指标与实现

离线排序报告 Spearman、NDCG@ K 和 Top- K 平均真实收益, $K \in \{5, 10, 18\}$ 。部署阶段报告 shortcut 数、平均展开节点变化、平均耗时、P95 耗时和距离正确率。在线变化率均相对同实现原图双向 Dijkstra 计算。G4 含 418,183 个参数, 采用 CUDA FP32、学习率 5×10^{-3} 、40 个 epoch、无 scheduler。训练只在 GPU 上进行。

C++ 评测在同一 C++20/O3 可执行程序内实现原图与压缩图双向 Dijkstra, 固定单 CPU 核, 预热 2 轮并正式重复 10 轮。该设计避免把不同语言或不同机器的计时差异误当作算法收益。

神经模型训练在 RTX 4090 D 上使用 CUDA; CPU 只承担数据审计、单元测试和最终 C++ 查询评测。均值与标准差按预先固定的全部种子报告。本文不把不同种子数的方法比较包装为显著性检验, 也不在观察 holdout 后删除异常种子。在线计时采用逐查询原图-压缩图配对, 查询顺序和实现保持一致; 算法无关的系统扰动通过完整干净复跑而不是选择性删除样本处理。

6 实验结果

6.1 RQ1: 历史需求传播能否预测区域价值?

Z0 在两城的全局 Spearman 上均高于 midpoint Proxy 与 MLP, 说明道路拓扑上的历史需求传播可以形成强区域价值轴。G4 相对 Z0 的 Porto 三种子增量为 $+0.012373/ +0.010313/ +0.008212$, Chicago 为 $+0.006405/ +0.006515/ +0.004035$, 六次比较方向一致。这一跨城市、跨种子的完全一致性表明, G4 学到的不是偶然扰动, 而是 Z0 之外可重复的全局排序信息。Chicago 的 NDCG@18 呈现不同方向, 则形成一组关键对照: soft Spearman 确实优化了完整候选序列, 而固定预算下的头部质量由另一目标决定。两类指标的分离不是全局增益失效, 而是学习目标边界被实验清楚识别。

6.2 RQ2: 冻结方法能否推广到未来窗口?

Porto 和 Chicago 的三个 G4 种子在 future Spearman 上均超过各自 Z0。结合空间 holdout, G4 在两城、三种子、两外部窗口的 12 次配对比较中全部取得正增量, 构成本文神经增强稳定性的核心证据。Porto 的 Y/F 区域收益 Spearman 高达 0.9959, 因此该结果对应稳定需求时段下的时间外泛化; 强分布漂移仍需专门实验。Chicago 的时间漂移更明显, G4 的全局增量更大, 但 Top- K 仍存在波动。

6.3 RQ3: Z0 的有效信号来自哪里?

度保持重连保留每个节点的入度和出度, 却破坏真实道路邻接; 两城四个切分均出现大幅下降, 支持“正确道路拓扑对全局区域价值轴具有独立贡献”。仅起点和仅终点均低于双向 Z0, 且终点场贡献更强。无向图的全局相关性接近 Z0, 但未来 NDCG@5 更低, 说明方向信息主要影响头部。保持两侧边际需求的 OD 重耦合几乎不改变结果, 因此现有证据支持边际需求传播, 不支持原始 OD 配对关系是必要因素。max pooling 几乎复现全局排序, mean pooling 明显较弱; 多深度读出在不同切分间更稳健, 但不存在对所有指标都最优的单一深度。

6.4 RQ4: 排序能否转化为精确在线收益?

直接按 Z0 分数选择 Porto Top-18 时, 候选并集只有 2,773 个唯一节点, 成员冗余系数为 3.323, 并存在共享 116 个节点的候选对, 无法由当前索引共同物化。hard-disjoint 贪心最终选满 18 个互不重叠区域, 覆盖 9,216 个唯一节点。该变化牺牲部分单区域平均收益, 却得到语义明确的可部署集合, 说明“预测高价值区域”和“组成高价值区域集合”是两个不同层次的问题。

Porto 中, G4 相对 Z0 仅多减少约 0.13/0.04 个百分点的当前/未来展开节点, 同时 shortcut 增加约 15.5%, 因此 Z0 的性能-存储性价比更高。Chicago 中, Z0 在展开节点上优于 G4 与 Proxy。两城既有和新增在线评测合计 204,000 个查询-索引配对, 距离正确率均为 100%。该结果再次表明, 全局排序增量需要经过部署评测后才能解释为系统收益。

表 2: 空间 holdout 排序结果。MLP 为五种子均值, G4 为三种子均值。

城市	方法	Spearman	NDCG@18
Porto	midpoint Proxy	0.891756	0.983040
	MLP	0.869552 $\pm$ 0.019732	0.959260 $\pm$ 0.014273
	Z0	0.935603	0.974533
	G4	0.945902 $\pm$ 0.001699	0.974551 $\pm$ 0.000110
Chicago	midpoint Proxy	0.895597	0.867650
	MLP	0.847990 $\pm$ 0.023984	0.901271 $\pm$ 0.028814
	Z0	0.941135	0.922494
	G4	0.946787 $\pm$ 0.001144	0.866699 $\pm$ 0.033993

表 3: 冻结未来窗口结果。

城市	方法	Spearman	NDCG@5	NDCG@18
Porto	Z0	0.944656	0.944421	0.933698
	G4	0.956598 $\pm$ 0.001580	0.893761 $\pm$ 0.005628	0.919343 $\pm$ 0.001397
Chicago	Z0	0.897509	0.659842	0.786935
	G4	0.925712 $\pm$ 0.000866	0.697910 $\pm$ 0.046648	0.765981 $\pm$ 0.030726

6.5 RQ5: 算法工作量减少是否转化为墙钟加速?

C++ 评测包含 560,000 个正式计时配对样本, 距离正确率为 100%, 并精确重放 Python 的区域与 shortcut。Porto 的展开与扫边同时减少, 因而平均耗时和 P95 均下降。Chicago 虽减少约 20% 的展开节点, 但密集边界 shortcut 使扫边增加 18%–28%, 抵消了典型查询的收益; 14 个方法-窗口组合中仍有 11 个 P95 下降。区域物化是否加速因此取决于边界形状与 shortcut 密度, 而不仅取决于区域排序精度。

7 讨论

7.1 确定性主方法与神经增强的分工

实验支持一个分层结论。Z0 无需标签和训练, 已经获得较强的全局排序与在线工作量收益, 适合作为部署主方法。G4 在两城、三种子和两个外部窗口组成的 12 次比较中全部提高全局 Spearman, 证明神经网络稳定提取了冻结先验未覆盖的排序信息。这使 G4 成为具有跨空间、跨时间重复证据的全局排序增强; 实际部署时, 再根据预算目标选择 Z0 或相应的神经增强版本。

7.2 全局排序与头部决策的目标分离及工程意义

本文的另一项重要发现是, 全局排序和头部选择并非同一目标的两种近似度量。Spearman 约束全部候选之间的相对次序, 而 NDCG@ K 和最终部署只关注预算截断点附近的少量候选; hard-disjoint 选择还会引入区域重叠和 shortcut 成本。G4 在 12/12 次比较中提高全局 Spearman, 而头部指标随城市和预算独立变化, 这一可重复分离实证证明了两类训练目标不可互相替代。该发现把原本容易被混为一谈的“排序更准”和“预算内选得更好”明确拆开, 也为后续预算感知损失和集合级学习提供了直接依据。

从工程角度看, 全局排序也不是脱离部署的“平均指标”。真实系统中的 K 会随存储和预处理预算变化, 高分区域可能因空间重叠、shortcut 过密或临时不可用而被淘汰, 此时系统必须持续从排序后部补充候选。只围绕一个固定 Top- K 优化, 容易把资源集中在少数当前热点, 并在预算或约束改变后降低不同预算间的决策复用性; 高质量的完整排序则为不同预算提供同一条可复用决策轴。结合本文的全图空间均匀候选和历史 OD 传播, 这条决策轴能够同时考虑繁忙区域与其余道路区域的实际需求-收益关系, 更适合全图资源调度。这里的“顾及全图”指工程覆盖与预算适应性, 不预设人口或行政意义上的公平结论。



图 2: 两城排序性能及全局指标与头部指标的差异。

表 4: 正确道路拓扑与度保持随机重连的 Spearman 对照。

城市与窗口	Z0	度保持随机重连	绝对下降
Porto holdout	0.935603	0.556581	0.379022
Porto future	0.944656	0.572467	0.372189
Chicago holdout	0.941169	0.605370	0.335799
Chicago future	0.897844	0.454814	0.443030

7.3 训练稳定性分析

早期 32 层网络曾出现约 22 个 epoch 不更新，随后突然变化。逐步诊断发现，FP16 的默认动态缩放导致早期 optimizer step 被跳过；在 FP32/BF16 下，梯度元素本身仍有限，但全局范数的 FP32 平方和发生溢出，裁剪系数继而接近零。32 层反向 hook 从未层约 10^{-6} 放大到首层约 10^{19} 。式(8)中的严格恒等路径使深层训练稳定。此后又发现旧 pairwise loss 与 Spearman 方向错位，最终才冻结 soft Spearman 协议。该过程说明“数值可训练”和“任务目标有效”是两个必须分别检验的问题。

7.4 适用范围与有效性

本文通过共享查询实现、时间窗口隔离、空间重叠组隔离和冻结后的多种子评测控制实验偏差；C++ 与 Python 的平均展开差异低于每查询 0.1 个节点。Spearman、NDCG、展开节点、扫描边和墙钟时间分别描述全局排序、头部质量、算法工作量与实现性能，因此本文按层报告，而不以单一指标替代端到端效果。12/12 次配对正增量为 G4 的全局排序能力提供了一致证据；Top-K 与在线收益由独立的预算目标和部署评测刻画。这一分层结论比用单一指标概括系统性能更完整。

Porto 出租车与 Chicago 共享单车提供了跨城市、跨出行方式证据，但尚未覆盖强需求突变、动态边权和更多路网密度。当前结论主要适用于存在重复 OD 查询、允许离线预处理且要求精确距离的静态道路分析任务。候选区域固定为 512 个节点，区域尺度、标签构造成本和边界 shortcut 密度仍会影响方法的性能-存储权

衡。

后续可进一步研究预算感知排序损失、边界 shortcut 稀疏化、动态需求更新和更多城市复现。对于 Chicago 一类密集边界图，将扫边成本直接纳入选区目标可能比单独提高全局 Spearman 更有效。

8 结论

本文在已有 CRP 式精确边界覆盖图上研究历史 OD 驱动的选择性区域物化。通过无路径输入的 Z0、以其为冻结先验的 G4，以及时间、空间和部署三层隔离协议，本文把离线区域排序与精确在线最短路查询连接为可审计闭环。两城结果表明，正确道路拓扑上的历史边际需求传播能够提供稳定且经济的区域价值排序；神经残差又在 12/12 次跨城市、跨种子、跨窗口比较中实现全局排序增益，形成明确且可重复的 AI 贡献。更重要的是，实验实证证明全局排序与预算约束头部选择是不同的学习目标：前者为预算变化、重叠淘汰和全图候补提供稳定决策轴，后者负责特定预算下的局部最优选择。本文进一步揭示 shortcut 密度如何决定排序增益能否转化为墙钟收益，由此同时给出确定性强基线、稳定神经增强和面向全图实际需求的分层部署规律，并始终保持最短路答案完全精确。

A 冻结配置摘要

声明

数据可用性: _____

表 5: $K = 18$ 严格非重叠部署的在线展开节点结果。负值表示相对原图减少。

城市	方法	Shortcuts	当前展开变化	未来展开变化
Porto	随机	7,986	−4.91%	−4.73%
	历史热点	7,235	−15.84%	−15.90%
	midpoint Proxy	8,307	−17.35%	−17.66%
	Z0	6,712	−17.46%	−17.76%
	G4 (三种子)	7,755	−17.59%	−17.80%
Chicago	随机	26,011	−4.40%	−4.07%
	历史热点	20,093	−17.37%	−17.95%
	midpoint Proxy	21,516	−20.30%	−21.08%
	Z0	23,378	−21.47%	−21.92%
	G4 (三种子)	20,718	−19.62%	−20.28%

表 6: C++20/O3 同实现的 $K = 18$ 性能变化。负值表示降低。

城市	方法	当前均值	未来均值	当前 P95	未来 P95	当前扫边	未来扫边
Porto	Z0	−6.51%	−7.48%	−4.72%	−6.24%	−6.67%	−6.91%
	G4	−6.41%	−8.90%	−5.56%	−8.96%	−5.85%	−6.05%
Chicago	Z0	+3.57%	+0.72%	−3.13%	−3.85%	+26.56%	+27.51%
	G4	+3.03%	+3.02%	−2.24%	−1.33%	+18.58%	+19.26%

注：G4 为三种子均值；表中省略标准差，完整数值保留在实验产物中。

代码可用性： _____
基金项目： _____
利益冲突： _____
作者贡献： _____
致谢： _____

参考文献

[1] Tangpeng Dan, Xiao Pan, Bolong Zheng, and Xiaofeng Meng. FAHL: An efficient labeling index for flow-aware shortest path querying in road networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pages 836–849. IEEE, 2025. doi: 10.1109/ICDE65448.2025.00068.

[2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[3] Divvy. Divvy system data: Historical trip data. <https://divvybikes.com/system-data>, 2022. Accessed 2026-07-31.

[4] Alexandros Efentakis, Dieter Pfoser, and Yannis Vassiliou. SALT: A unified framework for all shortest-path query variants on road networks, 2014.

[5] Fabrizio Frasca, Emanuele Rossi, Davide Eynard, Ben Chamberlain, Michael M. Bronstein, and Federico Monti. SIGN: Scalable inception graph neural networks, 2020.

图待补

Porto 与 Chicago 的展开节点变化、扫边变化、平均耗时和 P95 变化四联图。

图 3: 选择性物化在两城中的系统收益与 shortcut 扫边成本。

表 7: 正式实验的关键冻结配置。

项目	固定值
时间窗口	$H = [0, 0.35)$, $Y = [0.35, 0.70)$, $F = [0.70, 1.00)$
候选	每城 1,200 个, 每候选 512 节点, 候选种子 42
空间隔离	Jaccard 阈值 0.50 的重叠连通组, 840/180/180
标签查询	每城、每窗口、每候选 2,000 条, 抽样种子 42, 端点缓存关闭
Z0	正反向 lazy mean, 深度 1/2/4/8/16/32, 区域 mean+max
G4	32 层双塔, hidden dim 32, 残差尺度 0.01, 418,183 参数
优化	CUDA FP32, 学习率 5×10^{-3} , weight decay 0, 40 epoch, 无 scheduler
模型选择	seeds 42/43/44, validation Spearman, $\alpha \in \{0, 0.125, 0.25, 0.5, 1\}$
部署	hard-disjoint, $K = 5/10/18$; 主系统表报告 $K = 18$
C++ 计时	C++20/O3, 固定单核, 2 轮预热, 10 轮正式重复, 每组 2,000 查询

- [6] Rolf Jagerman, Xuanhui Wang, Honglei Zhuang, Zhen Qin, Michael Bendersky, and Marc Najork. Rax: Composable learning-to-rank using JAX. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3051–3060. ACM, 2022. doi: 10.1145/3534678.3539065.
- [7] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. DeeperGCN: All you need to train deeper GCNs, 2020.
- [8] Xiaohua Li, Bin Wang, Tao Qiu, Ge Yu, and Ning Wang. Constructing effective caches of shortest path queries on road networks. *IEEE Access*, 8: 193777–193789, 2020. doi: 10.1109/ACCESS.2020.3024664.
- [9] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://arxiv.org/abs/1707.01926>.
- [10] Luis Moreira-Matias, Michel Ferreira, Joao Mendes-Moreira, et al. Taxi service trajectory—prediction challenge, ECML PKDD 2015, 2013.
- [11] OpenStreetMap contributors. Openstreetmap. <https://www.openstreetmap.org/copyright>, 2026. Open Database License; accessed 2026-07-31.
- [12] Jeppe Rishede Thomsen, Man Lung Yiu, and Christian S. Jensen. Effective caching of shortest paths for location-based services. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 313–324. ACM, 2012. doi: 10.1145/2213836.2213872.
- [13] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 32–42, 2021.
- [14] Jingyi Wan, Yongyong Gao, Yong Ma, Kai Huang, Xiaofang Zhou, Christian S. Jensen, and Bolong Zheng. Workload-aware shortest path distance querying in road networks. In *2022 IEEE 38th International Conference on Data Engineer-*

- ing (ICDE)*, pages 2372–2384. IEEE, 2022. doi: 10.1109/ICDE53745.2022.00223.
- [15] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5453–5462. PMLR, 2018.
- [16] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural bellman–ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/f6a673f09493afcd8b129a0bcf1cd5bc-Abstract.html>.